

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 11_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 21.5

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 5

11

Output: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int min(int a, int b) {  
    return (a < b) ? a : b;  
}
```

```
int main() {  
    int n, amount;  
    scanf("%d", &n);  
  
    int coins[20];  
    for (int i = 0; i < n; i++)  
        scanf("%d", &coins[i]);
```

```
    scanf("%d", &amount);
```

```
    int dp[105];
```

```
    for (int i = 0; i <= amount; i++)  
        dp[i] = 1000;
```

```
    dp[0] = 0;
```

```
    for (int i = 1; i <= amount; i++) {
```

```

    for (int j = 0; j < n; j++) {
        if (coins[j] <= i)
            dp[i] = min(dp[i], dp[i - coins[j]] + 1);
    }
}

if (dp[amount] == 1000)
    printf("-1");
else
    printf("%d", dp[amount]);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight $wt[i]$ A profit $val[i]$ if delivered on time A delivery deadline $deadline[i]$ (in hours from now) Rajesh has a truck with maximum weight capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

100 200 150 120

10 20 15 15

5 3 4 2

30

Output: 470

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
void sort(int n, int val[], int wt[], int d[]) {  
    for (int i = 0; i < n-1; i++) {  
        for (int j = i+1; j < n; j++) {  
            if (d[i] > d[j]) {  
                int t;  
                t = d[i]; d[i] = d[j]; d[j] = t;  
                t = wt[i]; wt[i] = wt[j]; wt[j] = t;  
                t = val[i]; val[i] = val[j]; val[j] = t;  
            }  
        }  
    }  
}
```

```
int main() {  
    int n, W;  
    scanf("%d", &n);
```

```

int val[25], wt[25], d[25];

for (int i = 0; i < n; i++) scanf("%d", &val[i]);
for (int i = 0; i < n; i++) scanf("%d", &wt[i]);
for (int i = 0; i < n; i++) scanf("%d", &d[i]);

scanf("%d", &W);

sort(n, val, wt, d);

int dp[105] = {0};

for (int i = 0; i < n; i++) {
    for (int t = W; t >= wt[i]; t--) {
        if (t <= d[i]) {
            dp[t] = max(dp[t], dp[t - wt[i]] + val[i]);
        }
    }
}

int ans = 0;
for (int i = 0; i <= W; i++)
    if (dp[i] > ans)
        ans = dp[i];

printf("%d", ans);

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$, representing the values and weights of n items respectively, and an integer W representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W .

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n , the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W , the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

50 60

5 10

10

Output: 60

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```

int main() {
    int n, W;
    scanf("%d", &n);

    int val[105], wt[105];

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    int dp[1005] = {0};

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
        }
    }

    printf("%d", dp[W]);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you

have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting) and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m , where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to $n-1$, where 0 is the headquarters, 1 through $n-2$ are the hotels, and $n-1$ is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u , v , and t , indicating that the bus can go directly between locations u and v in t seconds (in either direction).

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4

0 1 10

1 2 20

2 3 30

3 4 40

4 6

0 1 1

0 2 1

0 3 1

1 2 1

1 3 1

2 3 1

Output: Case 1: 300

Case 2: 6

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define INF 1000000000
```

```
int n, m;
```

```
int dist[25][25];
```

```
int dp[1<<15][20];
```

```
int min(int a, int b) { return a < b ? a : b; }
```

```
int solve(int mask, int pos) {  
    if (mask == (1<<(n-2)) - 1)  
        return dist[pos][n-1];
```

```
    if (dp[mask][pos] != -1)  
        return dp[mask][pos];
```

```
    int ans = INF;
```

```
    for (int i = 1; i <= n-2; i++) {  
        if (!(mask & (1<<(i-1)))) {  
            ans = min(ans,  
                dist[pos][i] +  
                solve(mask | (1<<(i-1)), i));  
        }  
    }
```

```
    return dp[mask][pos] = ans;  
}
```

```
int main() {
```

```
    int caseNo = 1;
```

```
    while (scanf("%d %d", &n, &m) != EOF) {
```

```
        for (int i = 0; i < n; i++)  
            for (int j = 0; j < n; j++)  
                dist[i][j] = (i == j) ? 0 : INF;
```

```
        for (int i = 0; i < m; i++) {  
            int u, v, t;  
            scanf("%d %d %d", &u, &v, &t);  
            dist[u][v] = dist[v][u] = t;  
        }
```

```
        for (int k = 0; k < n; k++)  
            for (int i = 0; i < n; i++)  
                for (int j = 0; j < n; j++)
```

```
        if (dist[i][k] + dist[k][j] < dist[i][j])
            dist[i][j] = dist[i][k] + dist[k][j];

    memset(dp, -1, sizeof(dp));

    int ans = solve(0, 0);

    ans += solve(0, n-1);

    printf("Case %d: %d\n", caseNo++, ans);
}

return 0;
```

Status : Partially correct

Marks : 1.5/10